

UNIVERSIDADE FEDERAL DE LAVRAS - UFLA

Departamento de Ciência da Computação

GCC118 - Programação Matemática

Otimização Combinatória em Redes Sociais: Aplicações do Problema de Coloração de Vértices

Autor: Guilherme Lirio Miranda

Docentes: Mayron César O. Moreira e Andreza C. Beezão Moreira

Lavras, MG
Outubro de 2026

Índice

Fase 0: Proposta de Trabalho Prático **2**

Fase 1: Relatório de Análise Exploratória **3**

Fase 2: Modelagem Matemática **5**

Fase 0: Proposta de Trabalho Prático

Modalidade: Opção A - Problema real com dados públicos

1. Objetivo do Trabalho

O presente trabalho tem como objetivo modelar e solucionar um problema real de otimização combinatória, percorrendo o ciclo completo da Programação Matemática. A proposta foca em estruturar o problema sob a ótica da Programação Linear Inteira (PLI) para obter soluções exatas em instâncias reduzidas através de um solver. Por se tratar de um problema NP-difícil em geral, o objetivo prático se estende ao desenvolvimento de métodos heurísticos, como a coloração sequencial por grau ou por saturação. A etapa final consistirá na comparação experimental do desempenho (tempo de execução versus qualidade da solução) da modelagem exata frente às estratégias heurísticas.

2. Problema Escolhido: Coloração de Vértices

O problema selecionado do banco de sugestões é a **Coloração de Vértices**. Na teoria dos grafos, o objetivo da coloração clássica é atribuir uma cor a cada vértice de forma que nenhum par de vértices adjacentes (conectados por uma aresta) compartilhe a mesma cor, buscando sempre minimizar o total de cores utilizadas (determinação do Número Cromático).

No contexto da aplicação escolhida, o grafo representará uma rede social de desenvolvedores de software. Cada classe de cor será um **Conjunto Independente**: um grupo de usuários sem conexão direta entre seus membros. A minimização permitirá particionar a rede no menor número desses grupos. Essa interpretação estrutural não garante independência estatística entre usuários nem elimina possíveis influências indiretas em testes A/B.

3. Base de Dados e Viabilidade

Os dados foram extraídos da coleção **Stanford Large Network Dataset Collection** (SNAP), uma biblioteca acadêmica pública. O dataset selecionado é o **GitHub Social Network**, gerado a partir de extrações da API pública da plataforma. As principais características da instância são:

- **Estrutura Topológica:** Grafo não direcionado, adequado à formulação clássica de coloração de vértices.
- **Tamanho da Instância:** 37.700 vértices (representando desenvolvedores ativos) e 289.003 arestas (representando relacionamentos de seguidores mútuos).
- **Densidade:** Aproximadamente 0,0004, caracterizando uma rede esparsa.

Devido ao porte da base de dados, o modelo exato de PLI será processado utilizando amostras extraídas do grafo original (subgrafos menores). Nas próximas fases, as heurísticas serão avaliadas também no grafo completo, para investigar seu comportamento em maior escala. A quantidade e a extração das instâncias são especificadas na fase 2.

Fase 1: Relatório de Análise Exploratória

A presente seção descreve a coleta e o estudo inicial dos dados propostos para o trabalho. O foco desta etapa é comprovar o tamanho da instância, averiguar a qualidade dos dados contidos nos arquivos extraídos e avaliar a viabilidade prática de utilização dessa base nas próximas etapas de modelagem matemática.

1. Proveniência e Coleta dos Dados

A base de dados escolhida origina-se do repositório acadêmico **SNAP** (Stanford Large Network Dataset Collection) e é intitulada **MUSAE GitHub Social Network**. Os dados foram construídos a partir da API pública do GitHub. Cada nó (vértice) na rede representa um desenvolvedor, enquanto as arestas simbolizam uma relação não direcionada, indicando que os desenvolvedores se seguem mutuamente na plataforma.

Os arquivos utilizados para caracterizar a base são:

- **musae_git_edges.csv:** Contém a lista de arestas que define as conexões do grafo.
- **musae_git_target.csv:** Fornece o cadastro de vértices e seus rótulos; apenas os IDs são usados na coloração.
- **musae_git_features.json:** Dicionário de características dos nós, usado para conferir a correspondência dos IDs. Os atributos não entram no modelo.

2. Tamanho da Instância e Caracterização Básica

A análise exploratória do grafo completo revelou a seguinte estruturação:

- **Quantidade de Vértices (|V|):** 37.700 vértices.
- **Quantidade de Arestas (|E|):** 289.003 arestas válidas na rede.
- **Densidade da Rede:** A razão entre as arestas existentes e o máximo possível é aproximadamente 0,000407, indicando um grafo esparso.

O **grau médio** é aproximadamente 15,33, o **grau mediano** é 6 e o **grau máximo** é 9.458. Esses valores indicam heterogeneidade nos graus e a presença de um vértice muito conectado. Não foi realizado ajuste estatístico que permita concluir que a distribuição segue uma lei de potência (scale-free).

3. Verificação de Qualidade dos Dados

Uma rotina de validação verificou as seguintes propriedades dos arquivos:

- **Arestas Duplicadas e Laços (Self-loops):** Não foram encontrados. A ausência de duplicatas evita restrições repetidas; um laço tornaria a coloração própria inviável, pois exigiria que um vértice tivesse cor diferente de si mesmo.

- **Continuidade dos Identificadores (IDs):** Os IDs variam de 0 a 37.699, formando um intervalo contínuo e íntegro. O cadastro do target e as chaves do arquivo de atributos correspondem ao mesmo conjunto de vértices. Essa identificação simplifica a indexação das variáveis de decisão.
- **Nós Isolados:** Não foram encontrados vértices isolados. A contagem de componentes, realizada separadamente, confirmou uma única componente conexa. A ausência de isolados, sozinha, não demonstraria essa conectividade.

4. Viabilidade para as Próximas Fases

Na formulação de atribuição da fase 2, com n vértices, m arestas e K cores disponíveis, há (n+1)K variáveis binárias e n+nK+mK+(K-1) restrições. Para a rede completa e a cota ingênua K=n=37.700, seriam 1.421.327.700 variáveis e 10.895.413.100 restrições somente de incompatibilidade em arestas. Essa dimensão motiva a restrição de escopo para os testes exatos; não constitui uma medição de memória nem demonstra a impossibilidade de outras formulações.

A base é adequada à proposta da fase 0: a formulação exata será avaliada em subgrafos induzidos de tamanhos crescentes, enquanto o grafo completo permitirá investigar o desempenho dos métodos heurísticos previstos para a fase 3. O desempenho do solver nas amostras deverá ser medido, sem pressupor que toda instância pequena seja fácil ou que toda instância grande seja difícil.

As verificações confirmam a consistência estrutural dos arquivos utilizados. Elas não demonstram ausência de ruído na rede social observada. A análise de incertezas e a comparação completa entre os métodos serão desenvolvidas nas etapas posteriores.

Fase 2: Modelagem Matemática

Esta etapa formula a coloração mínima como Programação Linear Inteira e avalia sua resolução em 34 subgrafos da rede GitHub. São apresentados o modelo, a construção das instâncias, a escolha do orçamento de tempo por um piloto e os resultados obtidos com PuLP. A avaliação é exata para cada subgrafo; o número cromático da rede completa não é determinado.

1. Formulação exata

Considere um grafo simples, não direcionado e não vazio G = (V, E), com n = |V| e m = |E|. Seja C = {1,...,K} o conjunto de cores disponíveis. O parâmetro K é obtido por uma coloração gulosa válida; assim, $\chi(G) \leq K$, onde $\chi(G)$ é o número cromático.

A gulosa visita os vértices por grau decrescente, desempata pelo menor ID e atribui a menor cor não usada pelos vizinhos já coloridos. A coloração é salva e verificada; apenas seu número de cores limita o modelo. Ela não é fornecida como solução inicial ao solver.

1.1 Variáveis, objetivo e restrições

$x_{vc} \in \{0,1\}$: vale 1 se o vértice v recebe a cor c.
$y_c \in \{0,1\}$: vale 1 se a cor c está ativada.

$$\min z = \Sigma_{c \in C}\, y_c \qquad (1)$$

$$\Sigma_{c \in C}\, x_{vc} = 1 \quad \forall v \in V \qquad (2)$$

$$x_{vc} \leq y_c \quad \forall v \in V,\, c \in C \qquad (3)$$

$$x_{uc} + x_{vc} \leq 1 \quad \forall \{u,v\} \in E,\, c \in C \qquad (4)$$

$$y_c \geq y_{c+1} \quad c = 1,...,K\text{-}1 \qquad (5)$$

$$x_{vc},\, y_c \in \{0,1\} \qquad (6)$$

O objetivo (1) minimiza as cores ativadas. Somar x no objetivo não serviria, pois (2) torna essa soma constante e igual a n. A restrição (2) atribui exatamente uma cor a cada vértice; (3) ativa qualquer cor atribuída; (4) impede conflitos entre vizinhos; e (6) exige decisões binárias.

A restrição (5) ordena as ativações. Os nomes das cores são arbitrários: renomeá-las para ocupar as primeiras posições mantém a coloração e seu valor. Essa quebra de simetria elimina parte das representações equivalentes sem excluir o ótimo.

1.2 Exatidão e tamanho do modelo

Toda solução inteira viável define uma coloração própria por (2) e (4). Por (3), seu objetivo é pelo menos a quantidade de cores usadas; portanto, z $\geq$ χ(G). Reciprocamente, uma coloração ótima usa χ(G) $\leq$ K cores. É possível renomeá-las para satisfazer (5) e construir x e y viáveis com objetivo χ(G). Logo, o ótimo da PLI coincide com o número cromático.

Não é necessário exigir que toda cor ativada tenha vértices: no ótimo, uma ativação vazia poderia ser removida e as cores renomeadas, diminuindo o objetivo. Em uma execução interrompida, o programa distingue cores ativadas e efetivamente usadas. A validade da cota K é essencial; um limite arbitrariamente baixo poderia excluir o ótimo.

$N_{\text{variáveis}} = (n+1)K$

$N_{\text{restrições}} = n + nK + mK + (K\text{-}1)$

As contagens são anteriores ao presolve. As quatro parcelas de restrições correspondem às equações (2), (3), (4) e (5). Os domínios binários não são contados como linhas. A dimensão depende tanto de n quanto de m e K.

No triângulo, n = 3, m = 3 e K = 3: são 12 variáveis e 23 restrições. Como todos os pares de vértices são adjacentes, são necessárias três cores. Esse caso e outros grafos pequenos de ótimo conhecido verificam a tradução das equações para o programa.

1.3 Relaxação e interpretação dos limites

Para K $\geq$ 2, ao relaxar a integralidade, $x_{vc} = y_c = 1/K$ satisfaz as restrições e produz objetivo 1. Somando (3) para um vértice e usando (2), o objetivo também é pelo menos 1. Assim, a relaxação básica vale 1, mesmo quando a solução inteira precisa de várias cores. Isso ajuda a interpretar limites inferiores fracos; não significa que exista coloração inteira com uma cor.

Neste trabalho, ativação e incompatibilidade são escritas separadamente, permitindo tratar também vértices isolados. A demonstração desta seção justifica especificamente as equações adotadas.

Uma formulação exata pode ser interrompida antes de provar o ótimo. Além disso, resolver G[S] determina uma informação sobre aquele subgrafo, e não o valor exato de χ(G) para a rede completa. A escolha de subgrafos é a restrição de escopo adotada para esta etapa.

2. Construção das instâncias

A rede de origem contém 37.700 vértices e 289.003 arestas. Em resposta à observação do professor sobre a quantidade de instâncias, define-se uma bateria principal com **30 subgrafos distintos** e um conjunto exploratório com outros **4 subgrafos**. O triângulo é apenas um teste didático e não entra nessa contagem.

Conjunto	Vértices por instância	Seleção	Sementes	Total
Principal	25, 50, 100, 200 e 400	BFS e fronteira	3 por combinação	30
Exploratório	800 e 1.600	BFS	2 por tamanho	4

Os tamanhos crescentes permitem observar a evolução do modelo e do esforço de solução. Dois procedimentos de seleção permitem contrastar estruturas locais diferentes, e três sementes reduzem a dependência de uma única amostra por combinação. Trata-se de um desenho inicial de avaliação computacional, sem pretensão de representar estatisticamente todos os subgrafos da rede.

2.1 Seleção dos vértices e das arestas

BFS: sorteia-se uniformemente a raiz entre os IDs cadastrados. Uma busca em largura expande uma fila; a lista ordenada de vizinhos é embaralhada antes de cada expansão. A inclusão é interrompida ao alcançar n vértices.

Fronteira: sorteia-se a raiz e, a cada passo, escolhe-se uniformemente um vértice entre os vizinhos ainda não incluídos do conjunto atual. A fronteira é atualizada após cada inclusão.

Após selecionar S, são mantidas **todas as arestas originais com ambos os extremos em S**, formando G[S], um subgrafo induzido. Usar somente as arestas de visita produziria uma árvore e alteraria artificialmente a coloração. Na base conectada utilizada, os dois procedimentos produzem amostras conectadas.

A semente principal é 10000(a+1)+100(b+1)+r: a=0 indica BFS e a=1 indica fronteira; b=0,...,4 representa os cinco tamanhos; r=1,2,3 distingue as amostras. Nos extras, usa-se 30000+n+r. Cada JSON registra a raiz, a semente, os IDs originais e as arestas.

Os quatro extras pertencem à seleção exploratória inicial e já integravam a bateria de 10 s. Foram acrescentados após a observação de casos menores fáceis. Sua análise permanece exploratória. Todas as 34 instâncias vêm da mesma rede e podem compartilhar vértices; não são observações independentes nem amostras uniformes de todos os subconjuntos possíveis.

O cadastro de vértices é lido do target e as arestas do edges. Rótulos web/ML e features não definem cores ou restrições. A regeneração dos subgrafos é determinística para o ambiente documentado; os arquivos e seus hashes permitem conferir as entradas usadas.

3. Implementação e escolha do tempo

O código usa Python 3.12.14 e PuLP 4.2.0. A chamada `modelo.solve()` não recebe um solver. O PuLP selecionou COIN_CMD/CBC neste ambiente; apenas tempo e log foram configurados. O pacote cbcbox 2.929 e o executável CBC Devel, compilado em 23/03/2026, estão identificados no registro do ambiente. A máquina utiliza Windows 11 e AMD Ryzen 5 3600.

Os testes preliminares de 10 s foram preservados: 26 dos 34 casos tiveram ótimo e 8 terminaram sem solução inteira retornada. Em BFS_400_1, o log registrou zero nós enumerados; em BFS_1600_1, a relaxação foi interrompida. Esse diagnóstico motivou avaliar um orçamento maior, sem alterar o modelo ou fornecer solução inicial.

3.1 Piloto e regra de decisão

Antes das novas execuções, foram selecionados seis casos: dois resolvidos no preliminar (BFS_100_1 e FRONTEIRA_400_2), dois principais sem solução (BFS_200_1 e BFS_400_1) e um de cada tamanho extra (BFS_800_1 e BFS_1600_1). Cada caso foi executado uma vez em 30 e 60 s, sequencialmente.

A regra registrada foi escolher 60 s se aumentasse o número de soluções inteiras ou de ótimos em relação a 30 s; caso contrário, escolher 30 s. Se algum dos quatro casos principais continuasse sem solução em 60 s, testar também 120 s e adotar esse orçamento somente se houvesse ganho nas mesmas contagens.

Instância	30 s: estado; UB/LB	60 s: estado; UB/LB	120 s: estado; UB/LB
BFS_100_1	O; 8/8,00	O; 8/8,00	O; 8/8,00
FRONTEIRA_400_2	O; 4/4,00	O; 4/4,00	O; 4/4,00
BFS_200_1	TF; 8/7,00	TF; 8/7,00	TF; 8/7,00
BFS_400_1	TS; -/1,00	TS; -/1,00	TS; -/1,00
BFS_800_1	TS; -/1,00	TS; -/1,00	TS; -/1,00
BFS_1600_1	TS; -/-	TS; -/1,00	TS; -/1,00

O: ótimo comprovado; TF: limite de tempo com solução inteira; TS: limite de tempo sem solução inteira. Traço: informação não retornada. As pastas do piloto incluem os tempos efetivos e os logs.

Decisão: 30 s: 3 soluções e 2 ótimos; 60 s: 3 soluções e 2 ótimos; 120 s: 3 soluções e 2 ótimos. A aplicação da regra selecionou **30 segundos**. As 34 instâncias foram então executadas novamente com esse limite uniforme. O piloto foi direcionado pelo preliminar e reutiliza casos da bateria; justifica uma escolha prática, mas não prova que o orçamento seja ideal nem constitui validação independente.

Em BFS_1600_1, 60 e 120 s permitiram obter LB=1, indisponível em 30 s. Houve, portanto, informação adicional de limite, mas sem nova solução inteira ou prova de ótimo. A regra prioriza essas duas contagens e não a disponibilidade de LB.

4. Resultados da bateria principal

As tabelas 1 e 2 apresentam as 30 instâncias principais com limite de 30 s. n e m são vértices e arestas; K é a cota gulosa; Var. e Restr. são as dimensões antes do presolve. UB é o objetivo inteiro retornado e LB é o limite inferior. B025-1 abrevia BFS_025_1; F025-1 abrevia FRONTEIRA_025_1.

Gap (%) = 100(UB-LB)/|UB|, quando ambos existem. O, TF e TS têm o significado definido no piloto; traços não representam zero. t é a duração da chamada Python, em segundos, incluindo comunicação com o executável e tratamento do retorno.

Tabela 1: amostras por BFS

Inst.	n	m	K	Var.	Restr.	UB	LB	Gap %	Fim	t (s)
B025-1	25	53	5	130	419	5	5,0	0,0	O	0,17
B025-2	25	77	6	156	642	6	6,0	0,0	O	0,25
B025-3	25	30	3	78	192	3	3,0	0,0	O	0,09
B050-1	50	191	6	306	1501	6	6,0	0,0	O	0,99
B050-2	50	54	3	153	364	3	3,0	0,0	O	0,11
B050-3	50	175	6	306	1405	6	6,0	0,0	O	0,52
B100-1	100	496	8	808	4875	8	8,0	0,0	O	4,52
B100-2	100	279	6	606	2379	5	5,0	0,0	O	0,97
B100-3	100	267	5	505	1939	5	5,0	0,0	O	0,61
B200-1	200	1313	8	1608	12311	8	7,0	12,5	TF	31,01
B200-2	200	705	6	1206	5635	6	6,0	0,0	O	3,62
B200-3	200	274	4	804	2099	4	4,0	0,0	O	0,76
B400-1	400	2556	12	4812	35883	-	1,0	-	TS	34,63
B400-2	400	1626	8	3208	16615	8	8,0	0,0	O	23,94
B400-3	400	2405	10	4010	28459	-	1,0	-	TS	33,13

Nas 15 amostras BFS, 12 tiveram ótimo comprovado e 13 tiveram solução inteira retornada. Essa contagem considera uma execução por instância, sem selecionar a melhor execução entre o piloto e a bateria.

Em BFS_200_1, UB=8 e LB=7,0 dão gap de 12,5%. Há uma solução viável, mas sua otimalidade não foi certificada.

4.1 Verificação dos resultados

Cada solução retornada é conferida quanto à integralidade (tolerância de 10^{-5}), atribuição única, ativação das cores, ausência de conflitos em todas as arestas, ordem de y e consistência do objetivo. A declaração de ótimo exige término Optimal e solução validada. A coloração gulosa é armazenada separadamente e não substitui a solução do solver na coluna UB.

Os tempos de modelagem, da chamada e do executável são registrados separadamente. A modelagem inclui a gulosa e a construção das restrições; exclui leitura do JSON e exportação LP. O tempo do executável é extraído da linha Total time (Wallclock seconds) do log CBC, quando disponível. Os detalhes por instância constam em resumo.csv e resultado.json.

Tabela 2: amostras por fronteira

Inst.	n	m	K	Var.	Restr.	UB	LB	Gap %	Fim	t (s)
F025-1	25	26	3	78	180	3	3,0	0,0	O	0,10
F025-2	25	25	3	78	177	3	3,0	0,0	O	0,10
F025-3	25	30	3	78	192	3	3,0	0,0	O	0,11
F050-1	50	63	3	153	391	3	3,0	0,0	O	0,13
F050-2	50	58	3	153	376	3	3,0	0,0	O	0,12
F050-3	50	83	5	255	719	4	4,0	0,0	O	0,25
F100-1	100	130	4	404	1023	4	4,0	0,0	O	0,23
F100-2	100	132	3	303	798	3	3,0	0,0	O	0,18
F100-3	100	124	4	404	999	3	3,0	0,0	O	0,23
F200-1	200	278	4	804	2115	4	4,0	0,0	O	0,49
F200-2	200	357	5	1005	2989	5	5,0	0,0	O	1,27
F200-3	200	296	4	804	2187	4	4,0	0,0	O	0,68
F400-1	400	593	4	1604	4375	4	4,0	0,0	O	4,61
F400-2	400	541	5	2005	5109	4	4,0	0,0	O	4,27
F400-3	400	518	4	1604	4075	4	4,0	0,0	O	1,71

4.2 Comparação por tamanho e seleção

n	BFS: ótimos	BFS: mediana t	Fronteira: ótimos	Fronteira: mediana t
25	3/3	0,17	3/3	0,10
50	3/3	0,52	3/3	0,13
100	3/3	0,97	3/3	0,23
200	2/3	3,62	3/3	0,68
400	1/3	33,13	3/3	4,27

As medianas descrevem somente estas três amostras distintas por grupo; não estimam a variabilidade de repetidas execuções de uma mesma instância. Incluem também os términos por limite de tempo.

Para n=400, as amostras BFS têm entre 1.626 e 2.556 arestas; as de fronteira, entre 518 e 593. Assim, comparar somente n esconde diferenças no tamanho do modelo e na estrutura. Os resultados observados não demonstram que BFS sempre produza casos mais difíceis, pois também variam as sementes e as características dos subgrafos.

Em 4 das 30 instâncias principais, o solver retornou uma solução com menos cores ativadas que a cota gulosa K. Isso mostra que K não deve ser interpretado como ótimo. Nas demais instâncias resolvidas até a otimalidade com UB=K, a PLI forneceu a certificação que a gulosa, isoladamente, não oferece.

5. Extensão exploratória e limitações

As quatro instâncias maiores foram executadas com o mesmo orçamento da bateria principal. Elas ampliam o intervalo de tamanhos observado, mas não foram usadas para uma comparação equilibrada entre BFS e fronteira.

Instância	n / m	K	Var. / Restr.	Fim	UB / LB	Gap %
BFS_800_1	800 / 3889	13	10413 / 61769	TS	- / 1,0	-
BFS_800_2	800 / 3350	11	8811 / 46460	TS	- / 1,0	-
BFS_1600_1	1600 / 11015	12	19212 / 152991	TS	- / -	-
BFS_1600_2	1600 / 8533	11	17611 / 113073	TS	- / 1,0	-

Instância	Modelagem (s)	Chamada (s)	Executável (s)
BFS_800_1	0,76	54,55	48,50
BFS_800_2	0,57	42,67	38,86
BFS_1600_1	1,90	185,83	31,20
BFS_1600_2	1,63	138,36	90,98

Considerando as 34 instâncias, foram 27 casos com ótimo comprovado, 1 com solução inteira sem prova de ótimo e 6 sem solução inteira retornada. A ausência de solução devolvida não significa inviabilidade: cada instância possui a coloração gulosa válida com K cores.

No preliminar, BFS_1600_1 levou 78,98 s na chamada Python e 16,32 s segundo o log, apesar do limite de 10 s. As medidas abrangem operações diferentes e o backend também pode ultrapassar o limite antes de verificar a parada. Não foi instrumentada uma decomposição que permita atribuir toda a diferença à exportação. Por isso, o limite solicitado e os tempos medidos são apresentados separadamente.

A seleção de subgrafos é local, utiliza uma única rede e admite sobreposição. No piloto, houve uma execução por caso e orçamento; na bateria, uma nova execução por instância no limite escolhido. Não foram planejadas repetições para estimar a variabilidade temporal. Os resultados não sustentam generalização para outras redes, comparação de solvers ou afirmação de impossibilidade de resolver a rede completa.

O resultado com limite de tempo é um retrato da formulação e do ambiente utilizados. Uma solução inteira com limites distintos fornece um intervalo para o ótimo, não uma certificação de seu valor. Quando LB não é retornado, o gap permanece indisponível, mesmo que exista uma coloração gulosa conhecida.